

Practically Groovy: Reaching for each

Taking a "least surprise" approach to iteration

Skill Level: Introductory

Scott Davis (scott@thirstyhead.com)

Founder

ThirstyHead.com

14 Apr 2009

In this *Practically Groovy* installment, Scott Davis provides a dizzying array of ways to iterate through ... arrays. And lists. And files. And URLs. And on and on and on. The most impressive part is that Groovy provides a consistent mechanism for walking through all of those collections and more.

Iteration is fundamental to programming. You're constantly presented with things — a `List`, a `File`, a `JDBC ResultSet` — that you need to walk through item by item. The Java language nearly always gives you a way to step through whatever you need to, but the frustrating reality is that it doesn't offer a standard way to do so. Groovy's approach to iteration is one of the eminently practical ways in which Groovy programming parts ways with Java programming. Through a series of code examples (all available for [download](#)), you'll learn in this article how Groovy's versatile `each()` method lets you leave the Java language's iteration quirks behind.

Java iteration strategies

Suppose you have a `java.util.List` of programming languages. Listing 1 demonstrates the programmatic way to say "I know each of these" in the Java language:

Listing 1. Java list iteration

```
import java.util.*;
```

```
public class ListTest{
    public static void main(String[] args){
        List<String> list = new ArrayList<String>();
        list.add("Java");
        list.add("Groovy");
        list.add("JavaScript");

        for(Iterator<String> i = list.iterator(); i.hasNext();){
            String language = i.next();
            System.out.println("I know " + language);
        }
    }
}
```

Thanks to the `java.lang.Iterable` interface that most collections classes share, you can iterate through a `java.util.Set` or `java.util.Queue` in an identical way.

About this series

Groovy is a modern programming language that runs on the Java platform. It offers seamless integration with existing Java code while introducing dramatic new features like closures and metaprogramming. Put simply, Groovy is what the Java language would look like had it been written in the 21st century.

The key to incorporating any new tool into your development toolkit is knowing when to use it and when to leave it in the box. Groovy can be extremely powerful, but only when applied properly to appropriate scenarios. To that end, the [Practically Groovy](#) series explores the practical uses of Groovy, helping you learn when and how to apply them successfully.

Now suppose that the languages are stored in a `java.util.Map`. Trying to get an `Iterator` on a `Map` fails at compile time — `Maps` don't implement the `Iterable` interface. Luckily, you can call `map.keySet()` to return a `Set`, and then you are back in business. The minor difference slowed you down a bit, but it didn't stop you. You just need to remember that `Lists`, `Sets`, and `Queues` implement `Iterable`, but `Maps` don't — even though they are all in the same `java.util` package.

Now assume that the languages live in a `String` array. An array is a data structure, not a class. You can't call `.iterator()` on a `String` array, so you are forced to use a slightly different iteration strategy. Again, you are hindered but not completely obstructed, as you can see in Listing 2:

Listing 2. Java array iteration

```
public class ArrayTest{
    public static void main(String[] args){
        String[] list = {"Java", "Groovy", "JavaScript"};

        for(int i = 0; i < list.length; i++){
            String language = list[i];
            System.out.println("I know " + language);
        }
    }
}
```

```
    }
  }
}
```

But wait — what about the for-each syntax that Java 5 introduced (see [Resources](#))? It works for any class that implements `Iterable`, as well as arrays, as shown in Listing 3:

Listing 3. The Java language's for-each iteration

```
import java.util.*;

public class MixedTest{
    public static void main(String[] args){
        List<String> list = new ArrayList<String>();
        list.add("Java");
        list.add("Groovy");
        list.add("JavaScript");

        for(String language: list){
            System.out.println("I know " + language);
        }

        String[] list2 = {"Java", "Groovy", "JavaScript"};
        for(String language: list2){
            System.out.println("I know " + language);
        }
    }
}
```

So now you can iterate through arrays and collections (except for Maps) in the same way. But what if the languages are stored in a `java.io.File`? How about a JDBC `ResultSet`? An XML document? A `java.util.StringTokenizer`? In each case, you're forced to use a slightly different iteration strategy. This approach wasn't intentional — the different APIs were developed at different times by different developers — but the fact is that you must know half a dozen Java iteration strategies and, more important, the special cases in which to use them.

Eric S. Raymond writes about "The Rule of Least Surprise" in his book *The Art of Unix Programming* (see [Resources](#)). He says, "To design usable interfaces, it's best when possible not to design an entire new interface model. Novelty is a barrier to entry; it puts a learning burden on the user, so minimize it." Groovy's attitude toward iteration follows Raymond's advice. When it comes to walking through nearly any structure in Groovy, all you need to remember is `each()`.

List iteration in Groovy

To begin, I'll refactor the `List` example in [Listing 3](#) to Groovy. Rather than tossing the `List` over the fence to a for loop (which, by the way, doesn't feel very object-oriented, does it?), you simply call the `each()` method directly on the list and pass in a closure.

Create a file named `listTest.groovy` and add the code in Listing 4:

Listing 4. Groovy list iteration

```
def list = ["Java", "Groovy", "JavaScript"]
list.each{language->
    println language
}
```

The first line in Listing 4 is Groovy shortcut syntax for constructing a `java.util.ArrayList`. You can add `println list.class` to the script to verify this. Next, you simply call `each()` on the list and print out the `language` variable in the body of the closure. You named the `language` variable at the beginning of the closure with the `language->` statement. If you don't supply a variable name, Groovy supplies a default one named `it`. Type `groovy listTest` at the command prompt to run `listTest.groovy`.

Listing 5 shows two progressively shorter versions of the code in [Listing 4](#):

Listing 5. Iteration using Groovy's `it` variable

```
// shorter, using the default it variable
def list = ["Java", "Groovy", "JavaScript"]
list.each{ println it }

// shorter still, using an anonymous list
["Java", "Groovy", "JavaScript"].each{ println it }
```

Groovy allows you to use the `each()` method on arrays and Lists interchangeably. To change the `ArrayList` to a `String` array, add `as String[]` to the end of the line, as shown in Listing 6:

Listing 6. Groovy array iteration

```
def list = ["Java", "Groovy", "JavaScript"] as String[]
list.each{println it}
```

The pervasiveness of the `each()` method in Groovy, coupled with the shortcut getter syntax (`getClass()` and `class` are identical calls), allows you to write code that is both concise and expressive at the same time. For example, say that you want to display all of the public methods of a given class using reflection. Listing 7 shows an example:

Listing 7. Groovy reflection

```
def s = "Hello World"
println s
```

```
println s.class
s.class.methods.each{println it}

//output:
$ groovy reflectionTest.groovy
Hello World
class java.lang.String
public int java.lang.String.hashCode()
public volatile int java.lang.String.compareTo(java.lang.Object)
public int java.lang.String.compareTo(java.lang.String)
public boolean java.lang.String.equals(java.lang.Object)
...
```

The last line of the script calls the `getClass()` method. A `java.lang.Class` offers a `getMethods()` method that returns an array. By chaining all of this together and then calling `each()` on the resulting array of `Methods`, you accomplish quite a bit in a single line of code.

But the omnipresent `each()` method isn't limited to just `Lists` and arrays, as the Java for-each structure is. In the Java language, the story ends here. In Groovy, you're just getting warmed up.

Map iteration

As you saw earlier, in the Java language you can't iterate directly over a `Map`. In Groovy, it's not a problem, as Listing 8 shows:

Listing 8. Groovy map iteration

```
def map = ["Java":"server", "Groovy":"server", "JavaScript":"web"]
map.each{ println it }
```

To get your hands on the parts of the name/value pair, you can either use the implicit `getKey()` and `getValue()` methods, or you can explicitly name the variables at the start of the closure, as shown in Listing 9:

Listing 9. Getting the key and value from maps

```
def map = ["Java":"server", "Groovy":"server", "JavaScript":"web"]
map.each{
    println it.key
    println it.value
}

map.each{k,v->
    println k
    println v
}
```

As you can see, iterating over a `Map` is just as natural as iterating over any other

collection.

Before I move on to the next iteration example, you should know about one other bit of syntactic sugar regarding Maps in Groovy. Rather than calling `map.get("Java")` as you would in the Java language, you can shorten the call to `map.Java`, as shown in Listing 10:

Listing 10. Getting map values

```
def map = ["Java":"server", "Groovy":"server", "JavaScript":"web"]  
  
//identical results  
println map.get("Java")  
println map.Java
```

Although this Groovy shortcut syntax for Maps is undeniably cool, it is also the source of a common gotcha when you use reflection on Maps. A call to `list.class` yields `java.util.ArrayList`, whereas calling `map.class` returns `null`. This happens because the shortcut to getting a map element overrides the usual getter call. No element in the Map has a key of `class`, so the call correctly returns `null`, as in the example in Listing 11:

Listing 11. Groovy maps and null

```
def list = ["Java", "Groovy", "JavaScript"]  
println list.class  
// java.util.ArrayList  
  
def map = ["Java":"server", "Groovy":"server", "JavaScript":"web"]  
println map.class  
// null  
  
map.class = "I am a map element"  
println map.class  
// I am a map element  
  
println map.getClass()  
// class java.util.LinkedHashMap
```

This is a rare case of Groovy violating the Rule of Least Surprise, but because getting elements out of a map is far more common than using reflection, it is an exception that I can live with.

String iteration

Now that you are getting used to the `each()` method, it shows up in all kinds of interesting places. Suppose that you'd like to iterate over a `String`, letter by letter. The `each()` method is ready and waiting for you, as shown in Listing 12:

Listing 12. String iteration

```
def name = "Jane Smith"
name.each{letter->
    println letter
}
```

This opens up all sorts of possibilities, such as replacing all spaces with underscores, as shown in Listing 13:

Listing 13. Replacing spaces with underscores

```
def name = "Jane Smith"
println "replace spaces"
name.each{
    if(it == " "){
        print "_"
    }else{
        print it
    }
}

// output
Jane_Smith
```

Of course in the case of replacing a single letter, Groovy offers a more concise `replace` method. You could consolidate all of the code in Listing 13 to a single line: `"Jane Smith".replace(" ", "_")`. But for more-involved `String` manipulation, the `each()` method is the perfect solution.

Range iteration

Groovy offers a native `Range` type that practically begs to be iterated over. Anything separated by a double-dot (like `1..10`) is a `Range`. Listing 14 shows an example:

Listing 14. Range iteration

```
def range = 5..10
range.each{
    println it
}

//output:
5
6
7
8
9
10
```

Ranges are not limited to simple `Integers`. Consider the code in Listing 15, which

iterates over a Range of Dates:

Listing 15. Date iteration

```
def today = new Date()
def nextWeek = today + 7
(today..nextWeek).each{
    println it
}

//output:
Thu Mar 12 04:49:35 MDT 2009
Fri Mar 13 04:49:35 MDT 2009
Sat Mar 14 04:49:35 MDT 2009
Sun Mar 15 04:49:35 MDT 2009
Mon Mar 16 04:49:35 MDT 2009
Tue Mar 17 04:49:35 MDT 2009
Wed Mar 18 04:49:35 MDT 2009
Thu Mar 19 04:49:35 MDT 2009
```

As you can see, the `each( )` method shows up in precisely the places you would expect it to. The Java language lacks a native Range type, but it does offer a similar concept in the form of enums. And not surprisingly, `each( )` is ready and waiting for you there too.

Enumeration iteration

A Java enum is an arbitrary set of values that are stored in a specific order. Listing 16 shows how naturally the `each( )` method fits in with enums, just as it does with the Range operator:

Listing 16. enum iteration

```
enum DAY{
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY,
    FRIDAY, SATURDAY, SUNDAY
}

DAY.each{
    println it
}

(DAY.MONDAY..DAY.FRIDAY).each{
    println it
}
```

There are certain cases in Groovy where `each( )` isn't expressive enough as a method name. In the next several examples, you'll see the `each( )` method decorated in a way that's specific to the context it's used in. The Groovy `eachRow( )` method is a perfect example of this.

SQL iteration

When dealing with relational database tables, it's common to say, "I need to do something specific to each row in the table." Contrast this with the previous examples. You're more likely to say, "I need to do something specific to each language in the list." In this spirit, a `groovy.sql.Sql` object offers an `eachRow()` method, as shown in Listing 17:

Listing 17. ResultSet iteration

```
import groovy.sql.*

def sql = Sql.newInstance(
    "jdbc:derby://localhost:1527/MyDbTest;create=true",
    "username",
    "password",
    "org.apache.derby.jdbc.ClientDriver")

println("grab a specific field")
sql.eachRow("select name from languages"){ row ->
    println row.name
}

println("grab all fields")
sql.eachRow("select * from languages"){ row ->
    println("Name: ${row.name}")
    println("Version: ${row.version}")
    println("URL: ${row.url}\n")
}
```

The first line of this script instantiates a new `Sql` object by setting the JDBC connection string, username, password, and JDBC driver class. From there, you can call the `eachRow()` method, passing in the SQL select statement as a method argument. Inside the closure, you can reference the column names (`name`, `version`, `url`) as if there were actual `getName()`, `getVersion()`, and `getUrl()` methods.

This is significantly cleaner than the Java equivalent of having to spin up separate `DriverManagers`, `Connections`, `Statements`, and `JDBCResultSets`, and then having to clean them all up in a sea of nested `try/catch/finally` blocks.

In the case of the `Sql` object, you could argue that either `each()` or `eachRow()` would be a reasonable method name. But in the next few examples, I think that you'll agree that the name `each()` isn't nearly expressive enough.

File iteration

I wouldn't dream of iterating through a `java.io.File` line-by-line in raw Java code. By the time I finished with all of the nested `BufferedReaders` and `FileReaders`,

not to mention all of the exception handling at the tail end of the process, I'd have forgotten what I started out to do.

Listing 18 shows the whole process in the Java language:

Listing 18. Java file iteration

```
import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;

public class WalkFile {
    public static void main(String[] args) {
        BufferedReader br = null;
        try {
            br = new BufferedReader(new FileReader("languages.txt"));
            String line = null;
            while((line = br.readLine()) != null) {
                System.out.println("I know " + line);
            }
        } catch(FileNotFoundException e) {
            e.printStackTrace();
        } catch(IOException e) {
            e.printStackTrace();
        } finally {
            if(br != null) {
                try {
                    br.close();
                } catch(IOException e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
```

Listing 19 shows the equivalent process in Groovy:

Listing 19. Groovy file iteration

```
def f = new File("languages.txt")
f.eachLine{language->
    println "I know ${language}"
}
```

This is where the conciseness of Groovy really shines. I hope you can see now why I call Groovy ["A DSL for Java Programmers."](#)

Bear in mind that I'm dealing with the same `java.io.File` class in both Groovy and the Java language. If the file doesn't exist, the Groovy code will throw the same `FileNotFoundException` that the Java code does. The difference is that in Groovy there are no checked exceptions. It is my prerogative — not a language

requirement — to wrap the `eachLine()` structure in the `try/catch/finally` blocks. In a simple command-line script, I favor the brevity of the code in [Listing 19](#). If I'm performing the same iteration while running in an application server, I'm not nearly as cavalier about letting the exceptions go uncaught. I would wrap the `eachLine()` block in the same `try/catch` block as the Java version.

The `File` class offers several variations on the `each()` method. One of them is `splitEachLine(String separator, Closure closure)`. This means that you can not only iterate through the file line-by-line, but also split it into tokens at the same time. Listing 20 shows an example:

Listing 20. Splitting each line of a file

```
// languages.txt
// notice the space between the language and the version
Java 1.5
Groovy 1.6
JavaScript 1.x

// splitTest.groovy
def f = new File("languages.txt")
f.splitEachLine(" "){words->
    words.each{ println it }
}

// output
Java
1.5
Groovy
1.6
JavaScript
1.x
```

If you are dealing with binary files, Groovy also offers an `eachByte()` method.

Of course, a `File` in the Java language isn't always a file — sometimes it's a directory. Groovy offers several `each()` variations for dealing with subdirectories as well.

Directory iteration

Using Groovy as a shell-script (or batch-script) replacement is easy, thanks to how easily you can reach out and touch the file system. To get a directory listing of the current directory, see Listing 21:

Listing 21. Directory iteration

```
def dir = new File(".")
dir.eachFile{file->
    println file
}
```

The `eachFile()` method returns both files and subdirectories. Using the `isFile()` and `isDirectory()` methods available from the Java language, you can begin doing more-sophisticated things. Listing 22 shows an example:

Listing 22. Separating files and directories

```
def dir = new File(".")
dir.eachFile{file->
    if(file.isFile()){
        println "FILE: ${file}"
    }else if(file.isDirectory()){
        println "DIR:  ${file}"
    }else{
        println "Uh, I'm not sure what it is..."
    }
}
```

Because both Java methods return simple boolean values, you can tighten up this code with a Java ternary operator. Listing 23 shows an example:

Listing 23. Ternary operator

```
def dir = new File(".")
dir.eachFile{file->
    println file.isDirectory() ? "DIR:  ${file}" : "FILE: ${file}"
}
```

If you are only interested in directories, you can use `eachDir()` instead of `eachFile()`. There are also `eachDirMatch()` and `eachDirRecurse()` methods.

As you can see, a simple `each()` method for `File` wouldn't be nearly expressive enough. The semantics of the typical `each()` method are preserved on `Files`, but the method names are more descriptive to provide additional hints about the advanced functionality.

URL iteration

Once you understand how to iterate through a `File`, you can use the same principle to iterate through the response to an HTTP request. Groovy provides a convenient (and familiar) `eachLine()` method on `java.net.URL`.

For example, Listing 24 walks through the HTML of the `ibm.com` home page, line-by-line:

Listing 24. URL iteration

```
def url = new URL("http://www.ibm.com")
url.eachLine{line->
    println line
}
```

Of course, if this is all you want to do, Groovy provides a one-liner solution, thanks to the `toURL()` method it adds to all `Strings`:

```
"http://www.ibm.com".toURL().eachLine{ println it }.
```

But what if you want to do something more useful with the HTTP response? Specifically, what if the request you make is to a RESTful Web service, and the response contains XML that you'd like to parse? The `each()` method will help you out in that case as well.

XML iteration

You've seen how to use the `eachLine()` method on files and URLs. XML presents a slightly different problem — you are probably less interested in walking through an XML document line-by-line than walking through it element-by-element.

For example, say that your list of languages is stored in a file named `languages.xml`, shown in Listing 25:

Listing 25. The `languages.xml` file

```
<langs>
  <language>Java</language>
  <language>Groovy</language>
  <language>JavaScript</language>
</langs>
```

Groovy provides an `each()` method that you can use, but with a slight twist. If you parse the XML with a native Groovy class called `XmlSlurper`, you can `each()` your way through the elements. See Listing 26, for example:

Listing 26. XML iteration

```
def langs = new XmlSlurper().parse("languages.xml")
langs.language.each{
    println it
}

//output
Java
Groovy
JavaScript
```

The `langs.language.each` statement pulls out all of the elements under `<langs>` named `<language>`. If you had `<format>` and `<server>` elements in the mix as well, they wouldn't appear in the output of the `each()` method.

And if that isn't impressive enough, now pretend that this XML is available via a RESTful Web service instead of a file in the file system. Replace the path to the file with a URL, and the rest of the code remains unchanged, as shown in Listing 27:

Listing 27. XML iteration on a Web services call

```
def langs = new XmlSlurper().parse("http://somewhere.com/languages")
langs.language.each{
    println it
}
```

That's a pretty concise, nifty use of the `each()` method, wouldn't you say?

Conclusion

The best part of this whole `each()` exercise is that it cuts a huge swath across Groovy with very little effort. There isn't much novelty to iteration in Groovy once you know about the `each()` method. And as Raymond would say, that is exactly the point. Once you know how to iterate through a `List`, you instantly know how to iterate through an array, a `Map`, a `String`, a `Range`, an `enum`, an `SQL ResultSet`, a `File`, a directory, a URL, and even the elements of an XML document.

The last example in this article briefly touched on XML parsing with the `XmlSlurper`. Next time, I'll circle back around and show you how easy XML parsing is with Groovy. You'll see both the `XmlParser` and `XmlSlurper` in action, and get a better idea of why Groovy offers two similar yet different classes for XML parsing in the first place. Until then, I hope that you find plenty of practical uses for Groovy.

Downloads

Description	Name	Size	Download method
Source code for this article's examples	j-pg04149.zip	17KB	HTTP

[Information about download methods](#)

Resources

Learn

- [Practically Groovy](#) (Andrew Glover and Scott Davis, developerWorks): Read all the installments in this series.
- [Groovy](#): Learn more about Groovy at the project Web site.
- [AboutGroovy.com](#): Keep up with the latest Groovy news and article links.
- [Applying the Rule of Least Surprise](#): This chapter in *The Art of Unix Programming* (Eric Raymond, Addison-Wesley, 2003) explains the tao of doing the least surprising thing.
- ["Enhance looping in Java 5.0 with for/in"](#) (Brett McLaughlin, developerWorks, November 2004): The for/in (also known as for-each) loop was introduced in Java 5.0 as a convenience feature.
- [Mastering Grails](#): Scott Davis's companion series focuses on this Groovy-based platform for Web development.
- [Groovy Recipes](#) (Scott Davis, Pragmatic Programmers, 2008): Learn more about Groovy and Grails in Scott Davis' latest book.
- [Groovy mailing list](#): Browse, search, or subscribe to the Groovy mailing list.
- [Technology bookstore](#): Browse for books on these and other technical topics.
- [developerWorks Java technology zone](#): Find hundreds of articles about every aspect of Java programming.

Get products and technologies

- [Groovy](#): Download the latest Groovy ZIP file or tarball.

Discuss

- Check out [developerWorks blogs](#) and get involved in the [developerWorks community](#).

About the author

Scott Davis

Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of [ThirstyHead.com](#), a Groovy and Grails training company. His books include [Groovy Recipes: Greasing the Wheels of Java](#), [GIS for Web Developers: Adding Where to Your Application](#), [The Google Maps API](#), and [JBoss At Work](#). He writes two ongoing article series for IBM developerWorks: [Mastering Grails](#)

and *Practically Groovy*.

Trademarks

Java and all Java-based trademarks are trademarks of Sun Microsystems, Inc. in the United States, other countries, or both.